

 **练习题 2.44** 假设我们在对有符号值使用补码运算的 32 位机器上运行代码。对于有符号值使用的是算术右移，而对于无符号值使用的是逻辑右移。变量的声明和初始化如下：

```
int x = foo(); /* Arbitrary value */
int y = bar(); /* Arbitrary value */

unsigned ux = x;
unsigned uy = y;
```

对于下面每个 C 表达式，1) 证明对于所有的 x 和 y 值，它都为真(等于 1)；或者 2) 给出使得它为假(等于 0)的 x 和 y 的值：

- A. $(x > 0) \mid\mid (x-1 < 0)$
- B. $(x \& 7) != 7 \mid\mid (x < 29 < 0)$
- C. $(x * x) >= 0$
- D. $x < 0 \mid\mid -x <= 0$
- E. $x > 0 \mid\mid -x >= 0$
- F. $x+y == uy+ux$
- G. $x \sim y + uy * ux == -x$

2.4 浮点数

浮点表示对形如 $V = x \times 2^y$ 的有理数进行编码。它对执行涉及非常大的数字($|V| \gg 0$)、非常接近于 0($|V| \ll 1$)的数字，以及更普遍地作为实数运算的近似值的计算，是很有用的。

直到 20 世纪 80 年代，每个计算机制造商都设计了自己的表示浮点数的规则，以及对浮点数执行运算的细节。另外，它们常常不会太多地关注运算的精确性，而把实现的速度和简便性看得比数字精确性更重要。

大约在 1985 年，这些情况随着 IEEE 标准 754 的推出而改变了，这是一个仔细制订的表示浮点数及其运算的标准。这项工作是从 1976 年开始由 Intel 赞助的，与 8087 的设计同时进行，8087 是一种为 8086 处理器提供浮点支持的芯片。他们请 William Kahan(加州大学伯克利分校的一位教授)作为顾问，帮助设计未来处理器浮点标准。他们支持 Kahan 加入一个 IEEE 资助的制订工业标准的委员会。这个委员会最终采纳的标准非常接近于 Kahan 为 Intel 设计的标准。目前，实际上所有的计算机都支持这个后来被称为 IEEE 浮点的标准。这大大提高了科学应用程序在不同机器上的可移植性。

旁注 IEEE(电气和电子工程师协会)

电气和电子工程师协会(IEEE，读做“eye-triple-ee”)是一个包括所有电子和计算机技术的专业团体。它出版刊物，举办会议，并且建立委员会来定义标准，内容涉及从电力传输到软件工程。另一个 IEEE 标准的例子是无线网络的 802.11 标准。

在本节中，我们将看到 IEEE 浮点格式中数字是如何表示的。我们还将探讨舍入(rounding)的问题，即当一个数字不能被准确地表示为这种格式时，就必须向上调整或者向下调整。然后，我们将探讨加法、乘法和关系运算符的数学属性。许多程序员认为浮点数没意思，往坏了说，深奥难懂。我们将看到，因为 IEEE 格式是定义在一组小而一致的原则上的，所以它实际上是相当优雅和容易理解的。

2.4.1 二进制小数

理解浮点数的第一步是考虑含有小数值的二进制数字。首先，让我们来看看更熟悉的十进制表示法。十进制表示法使用如下形式的表示：

$$d_m d_{m-1} \cdots d_1 d_0 . d_{-1} d_{-2} \cdots d_{-n}$$

其中每个十进制数 d_i 的取值范围是 $0 \sim 9$ 。这个表达描述的数值 d 定义如下：

$$d = \sum_{i=-n}^m 10^i \times d_i$$

数字权的定义与十进制小数点符号（‘.’）相关，这意味着小数点左边的数字的权是 10 的正幂，得到整数值，而小数点右边的数字的权是 10 的负幂，得到小数值。例如，

$$12.34_{10} \text{ 表示数字 } 1 \times 10^1 + 2 \times 10^0 + 3 \times 10^{-1} + 4 \times 10^{-2} = 12 \frac{34}{100}。$$

类似，考虑一个形如

$$b_m b_{m-1} \cdots b_1 b_0 . b_{-1} b_{-2} \cdots b_{-n+1} b_{-n}$$

的表示法，其中每个二进制数字，或者称为位， b_i 的取值范围是 0 和 1，如图 2-31 所示。这种表示方法表示的数 b 定义如下：

$$b = \sum_{i=-n}^m 2^i \times b_i \quad (2.19)$$

符号 ‘.’ 现在变为了二进制的点，点左边的位的权是 2 的正幂，点右边的位的权是 2 的负幂。例如， 101.11_2 表示数字 $1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} +$

$$1 \times 2^{-2} = 4 + 0 + 1 + \frac{1}{2} + \frac{1}{4} = 5 \frac{3}{4}。$$

从等式 (2.19) 中可以很容易地看

出，二进制小数点向左移动一位相当于这个数被 2 除。例如， 101.11_2 表示数 $5 \frac{3}{4}$ ，而

10.111_2 表示数 $2 + 0 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} = 2 \frac{7}{8}$ 。类似，二进制小数点向右移动一位相当于将该

数乘 2。例如 1011.1_2 表示数 $8 + 0 + 2 + 1 + \frac{1}{2} = 11 \frac{1}{2}$ 。

注意，形如 $0.11 \cdots 1_2$ 的数表示的是刚好小于 1 的数。例如， 0.111111_2 表示 $\frac{63}{64}$ ，我们将用简单的表达法 $1.0 - \epsilon$ 来表示这样的数值。

假定我们仅考虑有限长度的编码，那么十进制表示法不能准确地表达像 $\frac{1}{3}$ 和 $\frac{5}{7}$ 这样的数。类似，小数的二进制表示法只能表示那些能够被写成 $x \times 2^y$ 的数。其他的值只能被近似地表示。例如，数字 $\frac{1}{5}$ 可以用十进制小数 0.20 精确表示。不过，我们并不能把它准确地表示为一个二进制小数，我们只能近似地表示它，增加二进制表示的长度可以提高表示的精度：

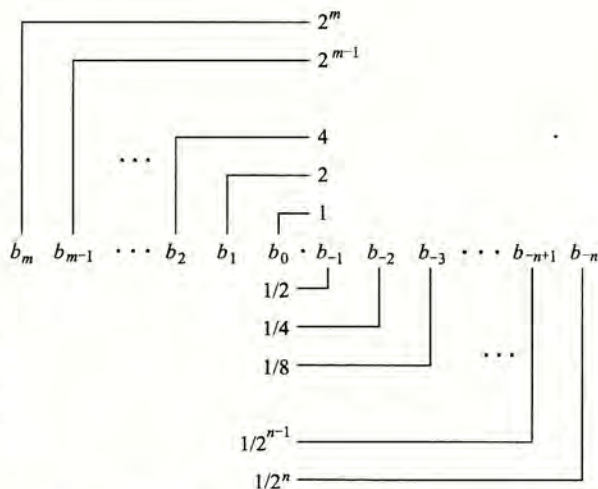


图 2-31 小数的二进制表示。二进制点左边的数字的权形如 2^i ，而右边的数字的权形如 $1/2^i$

表示	值	十进制
0.0_2	$\frac{0}{2}$	0.0_{10}
0.01_2	$\frac{1}{4}$	0.25_{10}
0.010_2	$\frac{2}{8}$	0.25_{10}
0.0011_2	$\frac{3}{16}$	0.1875_{10}
0.00110_2	$\frac{6}{32}$	0.1875_{10}
0.001101_2	$\frac{13}{64}$	0.203125_{10}
0.0011010_2	$\frac{26}{128}$	0.203125_{10}
0.00110011_2	$\frac{51}{256}$	0.19921875_{10}

 练习题 2.45 填写下表中的缺失的信息：

小数值	二进制表示	十进制表示
$\frac{1}{8}$	0.001	0.125
$\frac{3}{4}$		
$\frac{25}{16}$		
	10.1011	
	1.001	
		5.875
		3.1875

 练习题 2.46 浮点运算的不精确性能够产生灾难性的后果。1991 年 2 月 25 日，在第一次海湾战争期间，沙特阿拉伯的达摩地区设置的美国爱国者导弹，拦截伊拉克的飞毛腿导弹失败。飞毛腿导弹击中了美国的一个兵营，造成 28 名士兵死亡。美国总审计局(GAO)对失败原因做了详细的分析[76]，并且确定底层的原因在于一个数字计算不精确。在这个练习中，你将重现总审计局分析的一部分。

爱国者导弹系统中含有一个内置的时钟，其实现类似一个计数器，每 0.1 秒就加 1。为了以秒为单位来确定时间，程序将用一个 24 位的近似于 1/10 的二进制小数值来乘以这个计数器的值。特别地，1/10 的二进制表达式是一个无穷序列 0.000110011[0011]...₂，其中，方括号里的部分是无限重复的。程序用值 x 来近似地表示 0.1， x 只考虑这个序列的二进制小数点右边的前 23 位： $x=0.00011001100110011001100$ 。(参考练习题 2.51，里面有关于如何能够更精确地近似表示 0.1 的讨论。)

- A. $0.1-x$ 的二进制表示是什么？
- B. $0.1-x$ 的近似的十进制值是多少？
- C. 当系统初始启动时，时钟从 0 开始，并且一直保持计数。在这个例子中，系统已经运行了大约 100 个小时。程序计算出的时间和实际的时间之差为多少？
- D. 系统根据一枚来袭导弹的速率和它最后被雷达侦测到的时间，来预测它将在哪里出现。假定飞毛腿的速率大约是 2000 米每秒，对它的预测偏差了多少？

通过一次读取时钟得到的绝对时间中的轻微错误，通常不会影响跟踪的计算。相反，它应该依赖于两次连续的读取之间的相对时间。问题是爱国者导弹的软件已经升级，可以使用更精确的函数来读取时间，但不是所有的函数调用都用新的代码替换了。结果就是，跟踪软件一次读取用的是精确的时间，而另一次读取用的是不精确的时间[103]。

2.4.2 IEEE 浮点表示

前一节中谈到的定点表示法不能很有效地表示非常大的数字。例如，表达式 5×2^{100} 是用 101 后面跟随 100 个零的位模式来表示。相反，我们希望通过给定 x 和 y 的值，来表示形如 $x \times 2^y$ 的数。

IEEE 浮点标准用 $V = (-1)^s \times M \times 2^E$ 的形式来表示一个数：

- 符号(sign) s 决定这数是负数($s=1$)还是正数($s=0$)，而对于数值 0 的符号位解释作为特殊情况处理。
- 尾数(significand) M 是一个二进制小数，它的范围是 $1 \sim 2 - \epsilon$ ，或者是 $0 \sim 1 - \epsilon$ 。
- 阶码(exponent) E 的作用是对浮点数加权，这个权重是 2 的 E 次幂(可能是负数)。将浮点数的位表示划分为三个字段，分别对这些值进行编码：
- 一个单独的符号位 s 直接编码符号 s 。
- k 位的阶码字段 $\text{exp} = e_{k-1} \dots e_1 e_0$ 编码阶码 E 。
- n 位小数字段 $\text{frac} = f_{n-1} \dots f_1 f_0$ 编码尾数 M ，但是编码出来的值也依赖于阶码字段的值是否等于 0。

图 2-32 给出了将这三个字段装进字中两种最常见的格式。在单精度浮点格式(C 语言中的 float)中， s 、 exp 和 frac 字段分别为 1 位、 $k=8$ 位和 $n=23$ 位，得到一个 32 位的表示。在双精度浮点格式(C 语言中的 double)中， s 、 exp 和 frac 字段分别为 1 位、 $k=11$ 位和 $n=52$ 位，得到一个 64 位的表示。

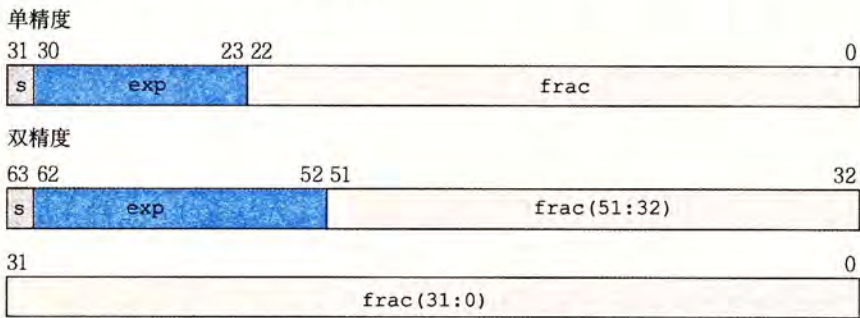


图 2-32 标准浮点格式(浮点数由 3 个字段表示。两种最常见的格式是它们被封装到 32 位(单精度)和 64 位(双精度)的字中)

给定位表示，根据 exp 的值，被编码的值可以分成三种不同的情况(最后一种情况有两个变种)。图 2-33 说明了对单精度格式的情况。

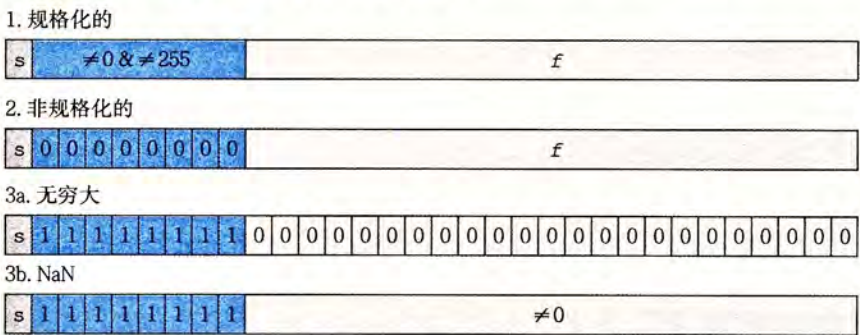


图 2-33 单精度浮点数值分类(阶码的值决定了这个数是规格化的、非规格化的或特殊值)

情况 1: 规格化的值

这是最普遍的情况。当 exp 的位模式既不全为 0 (数值 0), 也不全为 1 (单精度数值为 255, 双精度数值为 2047) 时, 都属于这类情况。在这种情况下, 阶码字段被解释为以偏置 (biased) 形式表示的有符号整数。也就是说, 阶码的值是 $E = e - \text{Bias}$, 其中 e 是无符号数, 其位表示为 $e_{k-1} \cdots e_1 e_0$, 而 Bias 是一个等于 $2^{k-1} - 1$ (单精度是 127, 双精度是 1023) 的偏置值。由此产生指数的取值范围, 对于单精度是 $-126 \sim +127$, 而对于双精度是 $-1022 \sim +1023$ 。

小数字段 frac 被解释为描述小数值 f , 其中 $0 \leq f < 1$, 其二进制表示为 $0.f_{n-1} \cdots f_1 f_0$, 也就是二进制小数点在最高有效位的左边。尾数定义为 $M = 1 + f$ 。有时, 这种方式也叫做隐含的以 1 开头的 (implied leading 1) 表示, 因为我们可以把 M 看成一个二进制表达式为 $1.f_{n-1} f_{n-2} \cdots f_0$ 的数字。既然我们总是能够调整阶码 E , 使得尾数 M 在范围 $1 \leq M < 2$ 之中 (假设没有溢出), 那么这种表示方法是一种轻松获得一个额外精度位的技巧。既然第一位总是等于 1, 那么我们就不需要显式地表示它。

情况 2: 非规格化的值

当阶码域为全 0 时, 所表示的数是非规格化形式。在这种情况下, 阶码值是 $E = 1 - \text{Bias}$, 而尾数的值是 $M = f$, 也就是小数字段的值, 不包含隐含的开头的 1。

旁注 对于非规格化值为什么要这样设置偏置值

使阶码值为 $1 - \text{Bias}$ 而不是简单的一 Bias 似乎是违反直觉的。我们将很快看到, 这种方式提供了一种从非规格化值平滑转换到规格化值的方法。

非规格化数有两个用途。首先, 它们提供了一种表示数值 0 的方法, 因为使用规格化数, 我们必须总是使 $M \geq 1$, 因此我们就不能表示 0。实际上, $+0.0$ 的浮点表示的位模式为全 0: 符号位是 0, 阶码字段全为 0 (表明是一个非规格化值), 而小数域也全为 0, 这就得到 $M = f = 0$ 。令人奇怪的是, 当符号位为 1, 而其他域全为 0 时, 我们得到值 -0.0 。根据 IEEE 的浮点格式, 值 $+0.0$ 和 -0.0 在某些方面被认为是不同的, 而在其他方面是相同的。

非规格化数的另外一个功能是表示那些非常接近于 0.0 的数。它们提供了一种属性, 称为逐渐溢出 (gradual underflow), 其中, 可能的数值分布均匀地接近于 0.0。

情况 3: 特殊值

最后一类数值是当指阶码全为 1 的时候出现的。当小数域全为 0 时, 得到的值表示无穷, 当 $s=0$ 时是 $+\infty$, 或者当 $s=1$ 时是 $-\infty$ 。当我们把两个非常大的数相乘, 或者除以零时, 无穷能够表示溢出的结果。当小数域为非零时, 结果值被称为 “NaN”, 即 “不是一个数 (Not a Number)” 的缩写。一些运算的结果不能是实数或无穷, 就会返回这样的 NaN 值, 比如当计算 $\sqrt{-1}$ 或 $\infty - \infty$ 时。在某些应用中, 表示未初始化的数据时, 它们也很有用处。

2.4.3 数字示例

图 2-34 展示了一组数值, 它们可以用假定的 6 位格式来表示, 有 $k=3$ 的阶码位和 $n=2$ 的尾数位。偏置量是 $2^{3-1} - 1 = 3$ 。图中的 a 部分显示了所有可表示的值 (除了 NaN)。两个无穷值在两个末端。最大数量值的规格化数是 ± 14 。非规格化数聚集在 0 的附近。图的 b 部分中, 我们只展示了介于 -1.0 和 $+1.0$ 之间的数值, 这样就能够看得更加清楚了。两个零是特殊的非规格化数。可以观察到, 那些可表示的数并不是均匀分布的——越靠近原点处它们越稠密。

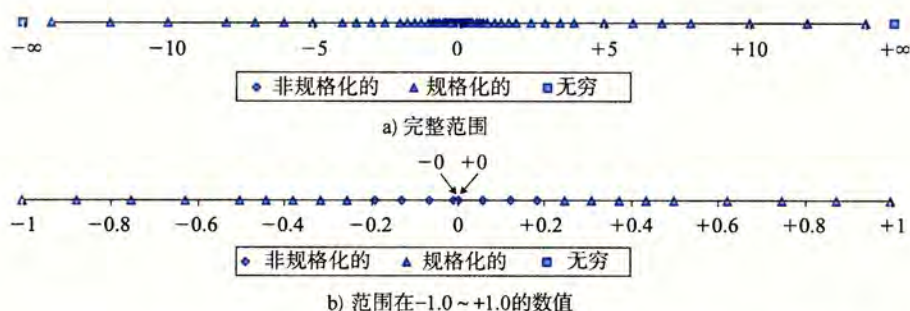


图 2-34 6 位浮点格式可表示的值($k=3$ 的阶码位和 $n=2$ 的尾数位。偏置量是 3)

图 2-35 展示了假定的 8 位浮点格式的示例，其中有 $k=4$ 的阶码位和 $n=3$ 的小数位。偏置量是 $2^{4-1}-1=7$ 。图被分成了三个区域，来描述三类数字。不同的列给出了阶码字段是如何编码阶码 E 的，小数字段是如何编码尾数 M 的，以及它们一起是如何形成要表示的值 $V=2^E \times M$ 的。从 0 自身开始，最靠近 0 的是非规格化数。这种格式的非规格化数的 $E=1-7=-6$ ，得到权 $2^E=\frac{1}{64}$ 。小数 f 的值的范围是 $0, \frac{1}{8}, \dots, \frac{7}{8}$ ，从而得到数 V 的范围是 $0 \sim \frac{1}{64} \times \frac{7}{8} = \frac{7}{512}$ 。

描述	位表示	指数			小数		值		
		e	E	2^E	f	M	$2^E \times M$	V	十进制
0 最小的非规格化数	0 0000 000	0	-6	$\frac{1}{64}$	$\frac{0}{8}$	$\frac{0}{8}$	$\frac{0}{512}$	0	0.0
	0 0000 001	0	-6	$\frac{1}{64}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{512}$	$\frac{1}{512}$	0.001953
	0 0000 010	0	-6	$\frac{1}{64}$	$\frac{2}{8}$	$\frac{2}{8}$	$\frac{2}{512}$	$\frac{2}{512}$	0.003906
	0 0000 011	0	-6	$\frac{1}{64}$	$\frac{3}{8}$	$\frac{3}{8}$	$\frac{3}{512}$	$\frac{3}{512}$	0.005859
	⋮								
最大的非规格化数	0 0000 111	0	-6	$\frac{1}{64}$	$\frac{7}{8}$	$\frac{7}{8}$	$\frac{7}{512}$	$\frac{7}{512}$	0.013672
1 最小的规格化数	0 0001 000	1	-6	$\frac{1}{64}$	$\frac{0}{8}$	$\frac{8}{8}$	$\frac{8}{512}$	$\frac{1}{64}$	0.015625
	0 0001 001	1	-6	$\frac{1}{64}$	$\frac{1}{8}$	$\frac{9}{8}$	$\frac{9}{512}$	$\frac{9}{512}$	0.017578
	⋮								
	0 0110 110	6	-1	$\frac{1}{2}$	$\frac{6}{8}$	$\frac{14}{8}$	$\frac{14}{16}$	$\frac{7}{8}$	0.875
	0 0110 111	6	-1	$\frac{1}{2}$	$\frac{7}{8}$	$\frac{15}{8}$	$\frac{15}{16}$	$\frac{15}{16}$	0.9375
	0 0111 000	7	0	1	$\frac{0}{8}$	$\frac{8}{8}$	$\frac{8}{8}$	1	1.0
	0 0111 001	7	0	1	$\frac{1}{8}$	$\frac{9}{8}$	$\frac{9}{8}$	$\frac{9}{8}$	1.125
	0 0111 010	7	0	1	$\frac{2}{8}$	$\frac{10}{8}$	$\frac{10}{8}$	$\frac{5}{4}$	1.25
	⋮								
最大的规格化数	0 1110 110	14	7	128	$\frac{6}{8}$	$\frac{14}{8}$	$\frac{1792}{8}$	224	224.0
最大的规格化数	0 1110 111	14	7	128	$\frac{7}{8}$	$\frac{15}{8}$	$\frac{1920}{8}$	240	240.0
无穷大	0 1111 000	—	—	—	—	—	—	∞	—

图 2-35 8 位浮点格式的非负值示例($k=4$ 的阶码位的和 $n=3$ 的小数位。偏置量是 7)

这种形式的最小规格化数同样有 $E=1-7=-6$ ，并且小数取值范围也为 $0, \frac{1}{8}, \dots, \frac{7}{8}$ 。然而，尾数在范围 $1+0=1$ 和 $1+\frac{7}{8}=\frac{15}{8}$ 之间，得出数 V 在范围 $\frac{8}{512}=\frac{1}{64}$ 和 $\frac{15}{512}$ 之间。

可以观察到最大非规格化数 $\frac{7}{512}$ 和最小规格化数 $\frac{8}{512}$ 之间的平滑转变。这种平滑性归功于我们对非规格化数的 E 的定义。通过将 E 定义为 $1-Bias$ ，而不是一 $Bias$ ，我们可以补偿非规格化数的尾数没有隐含的开头的 1。

当增大阶码时，我们成功地得到更大的规格化值，通过 1.0 后得到最大的规格化数。这个数具有阶码 $E=7$ ，得到一个权 $2^E=128$ 。小数等于 $\frac{7}{8}$ 得到尾数 $M=\frac{15}{8}$ 。因此，数值是 $V=240$ 。超出这个值就会溢出到 $+\infty$ 。

这种表示具有一个有趣的属性，假如我们将图 2-35 中的值的位表达式解释为无符号整数，它们就是按升序排列的，就像它们表示的浮点数一样。这不是偶然的——IEEE 格式如此设计就是为了浮点数能够使用整数排序函数来进行排序。当处理负数时，有一个小的难点，因为它们有开头的 1，并且它们是按照降序出现的，但是不需要浮点运算来进行比较也能解决这个问题(参见家庭作业 2.84)。

 **练习题 2.47** 假设一个基于 IEEE 浮点格式的 5 位浮点表示，有 1 个符号位、2 个阶码位($k=2$)和两个小数位($n=2$)。阶码偏置量是 $2^{k-1}-1=1$ 。

下表中列举了这个 5 位浮点表示的全部非负取值范围。使用下面的条件，填写表格中的空白项：

- e ：假定阶码字段是一个无符号整数所表示的值。
- E ：偏置之后的阶码值。
- 2^E ：阶码的权重。
- f ：小数值。
- M ：尾数的值。
- $2^E \times M$ ：该数(未归约的)小数值。
- V ：该数归约后的小数值。
- 十进制：该数的十进制表示。

写出 2^E 、 f 、 M 、 $2^E \times M$ 和 V 的值，要么是整数(如果可能的话)，要么是形如 $\frac{x}{y}$ 的小数，这里 y 是 2 的幂。标注为 “—” 的条目不用填。

位	e	E	2^E	f	M	$2^E \times M$	V	十进制
0 00 00								
0 00 01								
0 00 10								
0 00 11								
0 01 00								
0 01 01	1	0	1	$\frac{1}{4}$	$\frac{5}{4}$	$\frac{5}{4}$	$\frac{5}{4}$	1.25
0 01 10								
0 01 11								
0 10 00								
0 10 01								
0 10 10								
0 10 11								
0 11 00	—	—	—	—	—	—		—
0 11 01	—	—	—	—	—	—		—
0 11 10	—	—	—	—	—	—		—
0 11 11	—	—	—	—	—	—		—

0x00359141, 而单精度浮点数 3510593.0 的十六进制表示为 0x4A564504。推导出这个浮点表示, 并解释整数和浮点数表示的位之间的关系。

练习题 2.49

- A. 对于一种具有 n 位小数的浮点格式, 给出不能准确描述的最小正整数的公式(因为要想准确表示它需要 $n+1$ 位小数)。假设阶码字段长度 k 足够大, 可以表示的阶码范围不会限制这个问题。
- B. 对于单精度格式($n=23$), 这个整数的数字值是多少?

2.4.4 舍入

因为表示方法限制了浮点数的范围和精度, 所以浮点运算只能近似地表示实数运算。因此, 对于值 x , 我们一般想用一种系统的方法, 能够找到“最接近的”匹配值 x' , 它可以用期望的浮点形式表示出来。这就是舍入(rounding)运算的任务。一个关键问题是在两个可能值的中间确定舍入方向。例如, 如果我有 1.50 美元, 想把它舍入到最接近的美元数, 应该是 1 美元还是 2 美元呢? 一种可选择的方法是维持实际数字的下界和上界。例如, 我们可以确定可表示的值 x^- 和 x^+ , 使得 x 的值位于它们之间: $x^- \leq x \leq x^+$ 。IEEE 浮点格式定义了四种不同的舍入方式。默认的方法是找到最接近的匹配, 而其他三种可用于计算上界和下界。

图 2-37 举例说明了四种舍入方式, 将一个金额数舍入到最接近的整数美元数。向偶数舍入(round-to-even), 也被称为向最接近的值舍入(round-to-nearest), 是默认的方式, 试图找到一个最接近的匹配值。因此, 它将 1.40 美元舍入成 1 美元, 而将 1.60 美元舍入成 2 美元, 因为它们是最接近的整数美元值。唯一的设计决策是确定两个可能结果中间数值的舍入效果。向偶数舍入方式采用的方法是: 它将数字向上或者向下舍入, 使得结果的最低有效数字是偶数。因此, 这种方法将 1.5 美元和 2.5 美元都舍入成 2 美元。

方式	1.40	1.60	1.50	2.50	-1.50
向偶数舍入	1	2	2	2	-2
向零舍入	1	1	1	2	-1
向下舍入	1	1	1	2	-2
向上舍入	2	2	2	3	-1

图 2-37 以美元舍入为例说明舍入方式(第一种方法是舍入到一个最接近的值, 而其他三种方法向上或向下限定结果, 单位为美元)

其他三种方式产生实际值的确界(guaranteed bound)。这些方法在一些数字应用中是很有用的。向零舍入方式把正数向下舍入, 把负数向上舍入, 得到值 $\hat{x}$, 使得 $|\hat{x}| \leq |x|$ 。向下舍入方式把正数和负数都向下舍入, 得到值 x^- , 使得 $x^- \leq x$ 。向上舍入方式把正数和负数都向上舍入, 得到值 x^+ , 满足 $x \leq x^+$ 。

向偶数舍入初看上去好像是个相当随意的目标——有什么理由偏向取偶数呢? 为什么不始终把位于两个可表示的值中间的值都向上舍入呢? 使用这种方法的一个问题就是很容易假想到这样的情景: 这种方法舍入一组数值, 会在计算这些值的平均数中引入统计偏差。我们采用这种方式舍入得到的一组数的平均值将比这些数本身的平均值略高一些。相反, 如果我们总是把两个可表示值中间的数字向下舍入, 那么舍入后的一组数的平均值将比这些数本身的平均值略低一些。向偶数舍入在大多数现实情况中避免了这种统计偏差。

在 50% 的时间里，它将向上舍入，而在 50% 的时间里，它将向下舍入。

在我们不想舍入到整数时，也可以使用向偶数舍入。我们只是简单地考虑最低有效数字是奇数还是偶数。例如，假设我们想将十进制数舍入到最接近的百分位。不管用那种舍入方式，我们都将把 1.2349999 舍入到 1.23，而将 1.2350001 舍入到 1.24，因为它们不是在 1.23 和 1.24 的正中间。另一方面我们将把两个数 1.2350000 和 1.2450000 都舍入到 1.24，因为 4 是偶数。

相似地，向偶数舍入法能够运用在二进制小数上。我们将最低有效位的值 0 认为是偶数，值 1 认为是奇数。一般来说，只有对形如 $XX\cdots X.YY\cdots Y100\cdots$ 的二进制位模式的数，这种舍入方式才有效，其中 X 和 Y 表示任意位值，最右边的 Y 是要被舍入的位置。只有这种位模式表示在两个可能的结果正中间的值。例如，考虑舍入值到最近的四分之一的问題（也就是二进制小数点右边 2 位）。我们将 10.00011_2 ($2\frac{3}{32}$) 向下舍入到 10.00_2 (2)， 10.00110_2 ($2\frac{3}{16}$) 向上舍入到 10.01_2 ($2\frac{1}{4}$)，因为这些值不是两个可能值的正中间值。我们将 10.11100_2 ($2\frac{7}{8}$) 向上舍入成 11.00_2 (3)，而 10.10100_2 ($2\frac{5}{8}$) 向下舍入成 10.10_2 ($2\frac{1}{2}$)，因为这些值是两个可能值的中间值，并且我们倾向于使最低有效位为零。

 **练习题 2.50** 根据舍入到偶数规则，说明如何将下列二进制小数值舍入到最接近的二分之一（二进制小数点右边 1 位）。对每种情况，给出舍入前后的数字值。

- A. 10.010_2
- B. 10.011_2
- C. 10.110_2
- D. 11.001_2

 **练习题 2.51** 在练习题 2.46 中我们看到，爱国者导弹软件将 0.1 近似表示为 $x = 0.00011001100110011001100_2$ 。假设使用 IEEE 舍入到偶数方式来确定 0.1 的二进制小数点右边 23 位的近似表示 x' 。

- A. x' 的二进制表示是什么？
- B. $x' - 0.1$ 的十进制表示的近似值是什么？
- C. 运行 100 小时后，计算时钟值会有多少偏差？
- D. 该程序对飞毛腿导弹位置的预测会有多少偏差？

 **练习题 2.52** 考虑下列基于 IEEE 浮点格式的 7 位浮点表示。两个格式都没有符号位——它们只能表示非负的数字。

1. 格式 A
 - 有 $k=3$ 个阶码位。阶码的偏置值是 3。
 - 有 $n=4$ 个小数位。
2. 格式 B
 - 有 $k=4$ 个阶码位。阶码的偏置值是 7。
 - 有 $n=3$ 个小数位。

下面给出了一些格式 A 表示的位模式，你的任务是它们转换成格式 B 中最接近的值。如果需要，请使用舍入到偶数的舍入原则。另外，给出由格式 A 和格式 B 表示的位模式对应的数字的值。给出整数（例如 17）或者小数（例如 17/64）。

格式A		格式B	
位	值	位	值
011 0000	1	0111 000	1
101 1110			
010 1001			
110 1111			
000 0001			

2.4.5 浮点运算

IEEE 标准指定了一个简单的规则，来确定诸如加法和乘法这样的算术运算的结果。把浮点值 x 和 y 看成实数，而某个运算 $\odot$ 定义在实数上，计算将产生 $Round(x \odot y)$ ，这是对实际运算的精确结果进行舍入后的结果。在实际中，浮点单元的设计者使用一些聪明的小技巧来避免执行这种精确的计算，因为计算只要精确到能够保证得到一个正确的舍入结果就可以了。当参数中有一个是特殊值(如 -0 、 $-\infty$ 或 NaN)时，IEEE 标准定义了一些使之更合理的规则。例如，定义 $1/-0$ 将产生 $-\infty$ ，而定义 $1/+0$ 会产生 $+\infty$ 。

IEEE 标准中指定浮点运算行为方法的一个优势在于，它可以独立于任何具体的硬件或者软件实现。因此，我们可以检查它的抽象数学属性，而不必考虑它实际上是如何实现的。

前面我们看到了整数(包括无符号和补码)加法形成了阿贝尔群。实数上的加法也形成了阿贝尔群，但是我们必须考虑舍入对这些属性的影响。我们将 $x +^f y$ 定义为 $Round(x + y)$ 。这个运算的定义针对 x 和 y 的所有取值，但是虽然 x 和 y 都是实数，由于溢出，该运算可能得到无穷值。对于所有 x 和 y 的值，这个运算是可交换的，也就是说 $x +^f y = y +^f x$ 。另一方面，这个运算是不可结合的。例如，使用单精度浮点，表达式 $(3.14 + 1e10) - 1e10$ 求值得到 0.0 ——因为舍入，值 3.14 会丢失。另一方面，表达式 $3.14 + (1e10 - 1e10)$ 得出值 3.14 。作为阿贝尔群，大多数值在浮点加法下都有逆元，也就是说 $x +^f -x = 0$ 。无穷(因为 $+\infty - \infty = NaN$)和 NaN 是例外情况，因为对于任何 x ，都有 $NaN +^f x = NaN$ 。

浮点加法不具有结合性，这是缺少的最重要的群属性。对于科学计算程序员和编译器编写者来说，这具有重要的含义。例如，假设一个编译器给定了如下代码片段：

```
x = a + b + c;
y = b + c + d;
```

编译器可能试图通过产生下列代码来省去一个浮点加法：

```
t = b + c;
x = a + t;
y = t + d;
```

然而，对于 x 来说，这个计算可能会产生与原始值不同的值，因为它使用了加法运算的不同的结合方式。在大多数应用中，这种差异小得无关紧要。不幸的是，编译器无法知道在效率和忠实于原始程序的确切行为之间，使用者愿意做出什么样的选择。结果是，编译器倾向于保守，避免任何对功能产生影响的优化，即使是很轻微的影响。

另一方面，浮点加法满足了单调性属性：如果 $a \geq b$ ，那么对于任何 a 、 b 以及 x 的值，除了 NaN ，都有 $x + a \geq x + b$ 。无符号或补码加法不具有这个实数(和整数)加法的属性。

浮点乘法也遵循通常乘法所具有的许多属性。我们定义 $x *^f y$ 为 $Round(x \times y)$ 。这个运算在乘法中是封闭的(虽然可能产生无穷大或 NaN)，它是可交换的，而且它的乘法单位元

为 1.0。另一方面，由于可能发生溢出，或者由于舍入而失去精度，它不具有可结合性。例如，单精度浮点情况下，表达式 $(1e20 * 1e20) * 1e-20$ 求值为 $+\infty$ ，而 $1e20 * (1e20 * 1e-20)$ 将得出 $1e20$ 。另外，浮点乘法在加法上不具备分配性。例如，单精度浮点情况下，表达式 $1e20 * (1e20 - 1e20)$ 求值为 0.0，而 $1e20 * 1e20 - 1e20 * 1e20$ 会得出 NaN。

另一方面，对于任何 a 、 b 和 c ，并且 a 、 b 和 c 都不等于 NaN，浮点乘法满足下列单调性：

$$a \geq b \text{ 且 } c \geq 0 \Rightarrow a *^f c \geq b *^f c$$

$$a \geq b \text{ 且 } c \leq 0 \Rightarrow a *^f c \leq b *^f c$$

此外，我们还可以保证，只要 $a \neq \text{NaN}$ ，就有 $a *^f a \geq 0$ 。像我们先前所看到的，无符号或补码的乘法没有这些单调性属性。

对于科学计算程序员和编译器编写者来说，缺乏结合性和分配性是很严重的问题。即使为了在三维空间中确定两条线是否交叉而写代码这样看上去很简单的任务，也可能成为一个很大的挑战。

2.4.6 C 语言中的浮点数

所有的 C 语言版本提供了两种不同的浮点数据类型：float 和 double。在支持 IEEE 浮点格式的机器上，这些数据类型就对应于单精度和双精度浮点。另外，这类机器使用向偶数舍入的舍入方式。不幸的是，因为 C 语言标准不要求机器使用 IEEE 浮点，所以没有标准的方法来改变舍入方式或者得到诸如 -0 、 $+\infty$ 、 $-\infty$ 或者 NaN 之类的特殊值。大多数系统提供 include(‘.h’) 文件和读取这些特征的过程库，但是细节随系统不同而不同。例如，当程序文件中出现下列句子时，GNU 编译器 GCC 会定义程序常数 INFINITY(表示 $+\infty$) 和 NAN(表示 NaN)：

```
#define _GNU_SOURCE 1
#include <math.h>
```

 **练习题 2.53** 完成下列宏定义，生成双精度值 $+\infty$ 、 $-\infty$ 和 0：

```
#define POS_INFINITY
#define NEG_INFINITY
#define NEG_ZERO
```

不能使用任何 include 文件(例如 math.h)，但你能利用这样一个事实：双精度能够表示的最大的有限数，大约是 1.8×10^{308} 。

当在 int、float 和 double 格式之间进行强制类型转换时，程序改变数值和位模式的原则如下(假设 int 是 32 位的)：

- 从 int 转换成 float，数字不会溢出，但是可能被舍入。
- 从 int 或 float 转换成 double，因为 double 有更大的范围(也就是可表示值的范围)，也有更高的精度(也就是有效位数)，所以能够保留精确的数值。
- 从 double 转换成 float，因为范围要小一些，所以值可能溢出成 $+\infty$ 或 $-\infty$ 。另外，由于精确度较小，它还可能被舍入。
- 从 float 或者 double 转换成 int，值将会向零舍入。例如，1.999 将被转换成 1，而 -1.999 将被转换成 -1 。进一步来说，值可能会溢出。C 语言标准没有对这种情况指定固定的结果。与 Intel 兼容的微处理器指定位模式 $[10 \dots 00]$ (字长为 w 时的 $TMin_w$) 为整数不确定(integer indefinite)值。一个从浮点数到整数的转换，如果不能为该浮点数找到一个合理的整数近似值，就会产生这样一个值。因此，表达式 $(\text{int}) + 1e10$ 会得到 -21483648 ，即从一个正值变成了一个负值。

旁注 Ariane 5——浮点溢出的高昂代价

将大的浮点数转换成整数是一种常见的程序错误来源。1996年6月4日，Ariane 5火箭初次航行，一个错误便产生了灾难性的后果。发射后仅仅37秒钟，火箭偏离了它的飞行路径，解体并且爆炸。火箭上载有价值5亿美元的通信卫星。

后来的调查[73, 33]显示，控制惯性导航系统的计算机向控制引擎喷嘴的计算机发送了一个无效数据。它没有发送飞行控制信息，而是送出了一个诊断位模式，表明在将一个64位浮点数转换成16位有符号整数时，产生了溢出。

溢出的值测量的是火箭的水平速率，这比早先的Ariane 4火箭所能达到的速度高出了5倍。在设计Ariane 4火箭软件时，他们小心地分析了这些数字值，并且确定水平速率决不会超出一个16位数的表示范围。不幸的是，他们在Ariane 5火箭的系统中简单地重用了这一部分，而没有检查它所基于的假设。

 **练习题 2.54** 假定变量 x 、 f 和 d 的类型分别是 `int`、`float` 和 `double`。除了 f 和 d 都不能等于 $+\infty$ 、 $-\infty$ 或者 `NaN`，它们的值是任意的。对于下面每个 C 表达式，证明它总是为真(也就是求值为 1)，或者给出一个使表达式不为真的值(也就是求值为 0)。

- A. `x == (int)(double) x`
- B. `x == (int)(float) x`
- C. `d == (double)(float) d`
- D. `f == (float)(double) f`
- E. `f == -(-f)`
- F. `1.0/2 == 1/2.0`
- G. `d*d >= 0.0`
- H. `(f+d)-f == d`

2.5. 小结

计算机将信息编码为位(比特)，通常组织成字节序列。有不同的编码方式用来表示整数、实数和字符串。不同的计算机模型在编码数字和多字节数据中的字节顺序时使用不同的约定。

C 语言的设计可以包容多种不同字长和数字编码的实现。64 位字长的机器逐渐普及，并正在取代统治市场长达 30 多年的 32 位机器。由于 64 位机器也可以运行 32 位机器编译的程序，我们的重点就放在区分 32 位和 64 位程序，而不是机器本身。64 位程序的优势是可以突破 32 位程序具有的 4GB 地址限制。

大多数机器对整数使用补码编码，而对浮点数使用 IEEE 标准 754 编码。在位级上理解这些编码，并且理解算术运算的数学特性，对于想使编写的程序能在全部数值范围上正确运算的程序员来说，是很重要的。

在相同长度的无符号和有符号整数之间进行强制类型转换时，大多数 C 语言实现遵循的原则是底层的位模式不变。在补码机器上，对于一个 w 位的值，这种行为是由函数 `T2Uw` 和 `U2Tw` 来描述的。C 语言隐式的强制类型转换会出现许多程序员无法预计的结果，常常导致程序错误。

由于编码的长度有限，与传统整数和实数运算相比，计算机运算具有非常不同的属性。当超出表示范围时，有限长度能够引起数值溢出。当浮点数非常接近于 0.0，从而转换成零时，也会下溢。

和大多数其他程序语言一样，C 语言实现的有限整数运算和真实的整数运算相比，有一些特殊的属性。例如，由于溢出，表达式 `x*x` 能够得出负数。但是，无符号数和补码的运算都满足整数运算的许多其他属性，包括结合律、交换律和分配律。这就允许编译器做很多的优化。例如，用 `(x<<3)-x` 取代表达式 `7*x` 时，我们就利用了结合律、交换律和分配律的属性，还利用了移位和乘以 2 的幂之间的关系。

我们已经看到了几种使用位级运算和算术运算组合的聪明方法。例如，使用补码运算，`~x+1` 等价于 `-x`。另外一个例子，假设我们想要一个形如 `[0, ..., 0, 1, ..., 1]` 的位模式，由 $w-k$ 个 0 后面紧跟着 k